

EIF200 – Fundamentos de Informática

Tutoría #2 - Ciclo II 2026

Martes 4 de agosto de 2026

Tutor:

Cipriano Rivera Escobar

Índice

1. Variables	2
1.1. ¿Qué es una variable?	2
1.2. ¿Para qué sirven las variables?	2
1.3. Características de una variable	2
2. Tipos de datos	2
3. Asignación de variables	3
4. Constantes	3
5. Operadores	4
5.1. Operadores matemáticos	4
5.2. Operadores de asignación	4
5.3. Operadores relacionales	5
5.4. Operadores lógicos	5
6. Estructuras de control	5
7. Estructura switch	8
7.1. Ejemplo	9
8. Estructuras iterativas (Ciclos)	10
8.1. Clasificación de los ciclos	10
8.2. Ciclos definidos	10
8.3. Ciclos indefinidos	10
9. Ejemplo	11
10. Ejercicios	11
10.1. Ejercicio 1	11
10.2. Ejercicio 2	11
10.3. Ejercicio 3	12
10.4. Ejercicio 4	12
10.5. Ejercicio 5	12
10.6. Ejercicio 6	12
10.7. Ejercicio 7	12
10.8. Ejercicio 8	12
10.9. Ejercicio 9	13
10.10. Ejercicio 10	13

1. Variables

1.1. ¿Qué es una variable?

Una variable es un espacio en memoria que permite almacenar información durante la ejecución de un programa.

El valor de una variable puede cambiar conforme el programa se ejecuta, por eso recibe el nombre de variable.

1.2. ¿Para qué sirven las variables?

Las variables se utilizan para:

- Almacenar información.
- Realizar cálculos.
- Guardar resultados.
- Utilizar datos ingresados por el usuario.

1.3. Características de una variable

Toda variable posee las siguientes características:

- Un nombre o identificador.
- Un tipo de dato.
- Un valor.

2. Tipos de datos

Una vez creada una variable, es necesario indicar qué tipo de información almacenará. A esto se le conoce como **tipo de dato**.

Los tipos de datos más utilizados en C++ son:

Tipo	Descripción
short	Almacena números enteros pequeños.
int	Almacena números enteros.
long	Almacena números enteros de mayor tamaño.
float	Almacena números decimales.
double	Almacena números decimales con mayor precisión.
char	Almacena un único carácter.
bool	Almacena valores lógicos: verdadero o falso.
string	Almacena cadenas de caracteres o texto.
void	Representa la ausencia de un tipo de dato o de un valor de retorno.

Cuadro 1: Tipos de datos básicos en C++.

3. Asignación de variables

Una vez declarada una variable, es posible asignarle un valor.

Es importante que el valor asignado corresponda con el tipo de dato de la variable.

Por ejemplo:

- Una variable de tipo **int** almacena números enteros.
- Una variable de tipo **float** o **double** almacena números decimales.
- Una variable de tipo **char** almacena un solo carácter.
- Una variable de tipo **bool** almacena únicamente los valores **true** o **false**.
- Una variable de tipo **string** almacena texto.

4. Constantes

Una constante es un dato cuyo valor permanece fijo durante toda la ejecución de un programa.

A diferencia de una variable, una constante no puede modificarse una vez que se le ha asignado un valor.

Las constantes se utilizan para representar valores que no cambian, por ejemplo:

- El valor de π .
- La cantidad de días de una semana.
- El porcentaje del IVA.

5. Operadores

Los operadores son símbolos que permiten realizar operaciones o comparaciones sobre los datos almacenados en las variables.

En C++ existen diferentes tipos de operadores:

- Operadores matemáticos.
- Operadores de asignación.
- Operadores relacionales.
- Operadores lógicos.

5.1. Operadores matemáticos

Los operadores matemáticos permiten realizar operaciones aritméticas entre valores.

Operador	Operación	Ejemplo
+	Suma	$5 + 3 = 8$
-	Resta	$5 - 3 = 2$
*	Multiplicación	$5 * 3 = 15$
/	División	$6 / 3 = 2$
%	Módulo o residuo	$5 \% 2 = 1$

Cuadro 2: Operadores matemáticos.

5.2. Operadores de asignación

Los operadores de asignación permiten guardar o modificar el valor de una variable. El operador principal es:

- `=` : asigna un valor a una variable.

También existen operadores de asignación compuesta:

- `+=` : suma un valor y lo asigna.
- `-=` : resta un valor y lo asigna.
- `*=` : multiplica un valor y lo asigna.
- `/=` : divide un valor y lo asigna.

5.3. Operadores relacionales

Los operadores relacionales permiten comparar dos valores.
El resultado de una comparación siempre será un valor lógico:

- Verdadero (**true**).
- Falso (**false**).

Operador	Significado
==	Igual que
!=	Diferente de
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que

Cuadro 3: Operadores relacionales.

5.4. Operadores lógicos

Los operadores lógicos permiten combinar varias condiciones dentro de un programa.
Los principales operadores son:

Operador	Significado
&&	AND: ambas condiciones deben cumplirse.
	OR: al menos una condición debe cumplirse.
!	NOT: invierte el resultado de una condición.

Cuadro 4: Operadores lógicos.

6. Estructuras de control

Las estructuras de control permiten que un programa tome decisiones dependiendo del resultado de una condición.

Cuando una condición se cumple, el programa ejecuta un conjunto de instrucciones; en caso contrario, puede ejecutar otras diferentes.

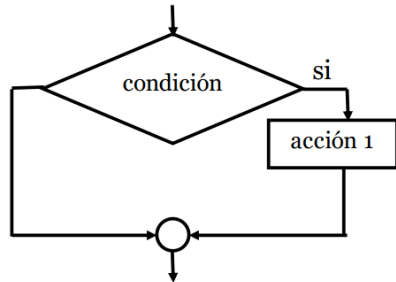
Las estructuras de control se clasifican en:

- Alternativa simple.
- Alternativa doble.
- Alternativa múltiple.

Estructuras de control

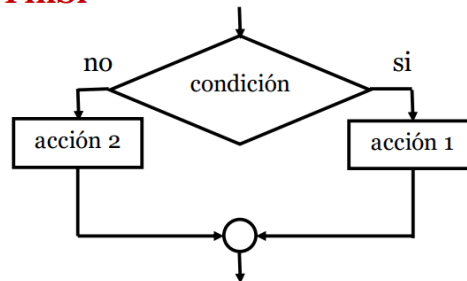
Alternativa simple

Si <condición> **Entonces**
 <acción 1>
FinSi



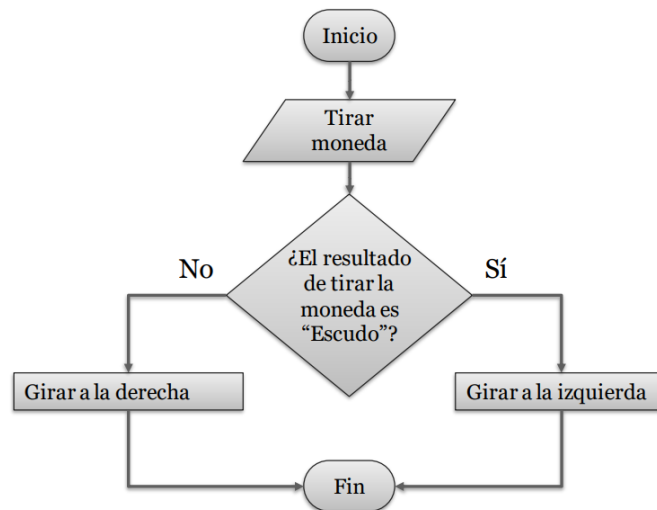
Alternativa doble

Si <condición> **Entonces**
 <acción 1>
Sino
 <acción 2>
FinSi



EIF 200 FUNDAMENTOS DE INFORMÁTICA

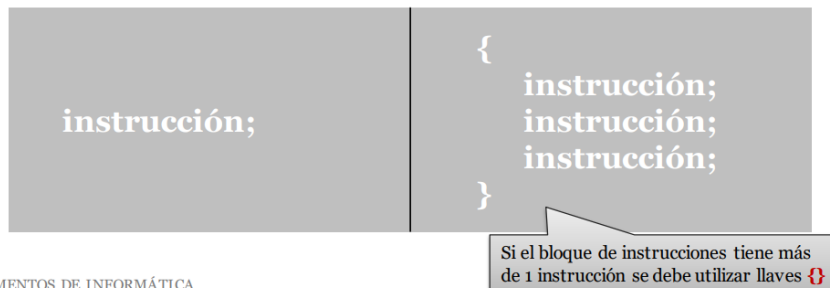
Ejemplo



Sintaxis C++

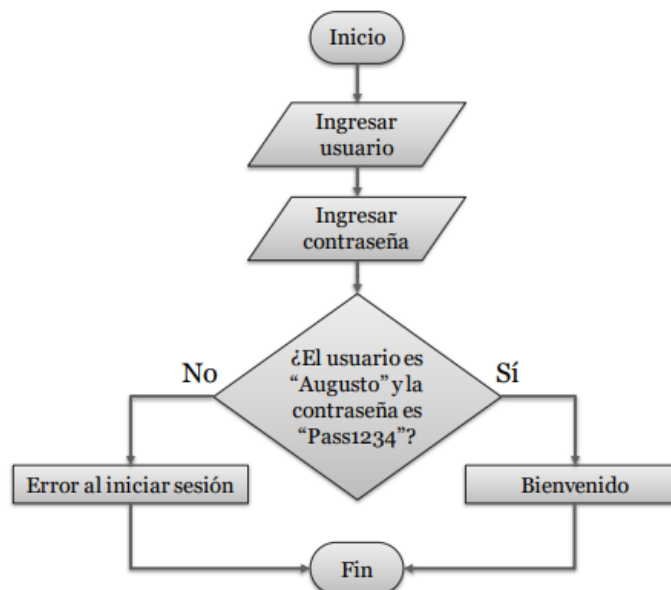
□ **if** (<expresión condicional>)
 <bloque de instrucciones>
else
 <bloque de instrucciones>

Donde: <bloque de instrucciones> puede ser:



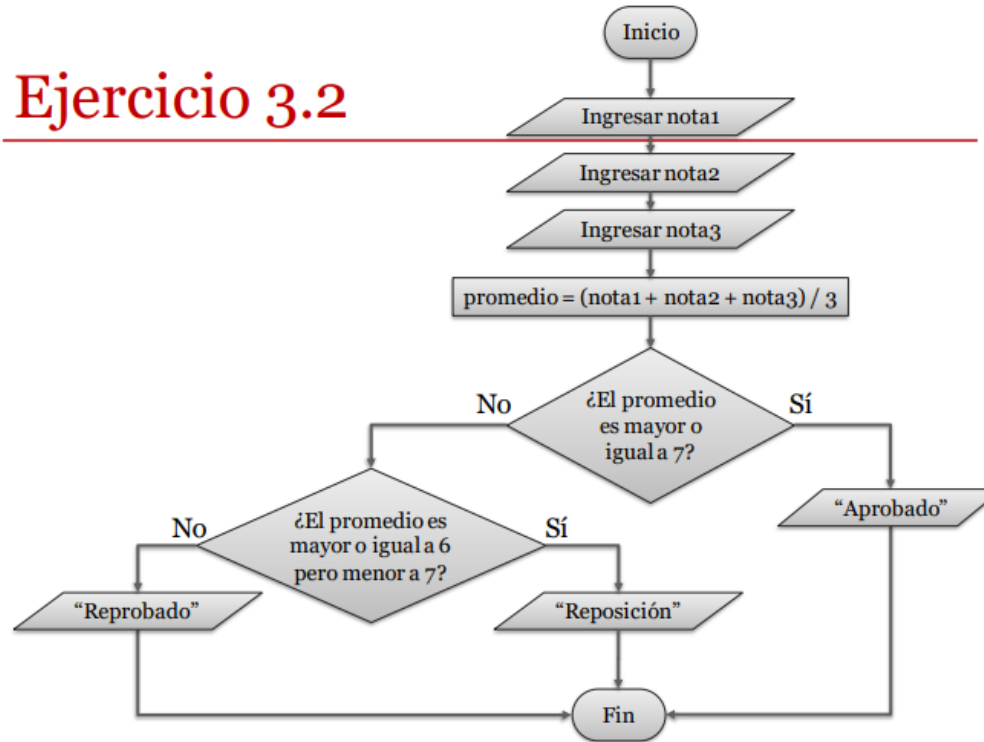
EIF 200 FUNDAMENTOS DE INFORMÁTICA

Ejercicio 3.1



EIF 200 FUNDAMENTOS DE INFORMÁTICA

Ejercicio 3.2



EIF 200 FUNDAMENTOS DE INFORMÁTICA

7. Estructura switch

La estructura **switch** es una estructura de control condicional que permite definir múltiples casos que puede llegar a cumplir una variable cualquiera y determinar la acción que se debe ejecutar en cada uno de ellos.

Además, es posible indicar una acción que se ejecutará cuando la variable no coincida con ninguno de los casos definidos.

Se utiliza principalmente cuando una misma variable puede tomar diferentes valores y cada uno requiere una acción distinta.

Sintaxis C++

□ switch (<variable>)

```
case valor1:
    <bloque de instrucciones>;
    break;

case valor2:
    <bloque de instrucciones>;
    break;

case valor3:
    <bloque de instrucciones>;
    break;

default:
    <bloque de instrucciones>;
```

El **valor(1, 2, 3, ...)** son los valores que puede tomar la <variable>

La instrucción **break** es necesario para no ejecutar todos los demás casos

El bloque de **default** se ejecuta en caso de que ningún otro caso se cumpla

EIF 200 FUNDAMENTOS DE INFORMÁTICA

7.1. Ejemplo

```
#include <iostream>

int main() {

    int opcion;

    std::cout << "Seleccione la opcion: ";
    std::cin >> opcion;

    switch(opcion)
    {
        case 1:
            // Guardar juego
            break;

        case 2:
            // Cargar partida
            break;

        case 3:
            // Definir jugadores
            break;
```

```
        default:
            // Volver al menu principal
            break;
    }

    return 0;
}
```

8. Estructuras iterativas (Ciclos)

Las estructuras iterativas, también conocidas como ciclos o bucles, permiten ejecutar un mismo bloque de instrucciones varias veces mientras se cumpla una condición o durante un número determinado de repeticiones.

Su principal objetivo es evitar escribir el mismo código varias veces, haciendo los programas más cortos, organizados y fáciles de mantener.

8.1. Clasificación de los ciclos

Los ciclos se pueden clasificar en dos grupos:

- Ciclos definidos.
- Ciclos indefinidos.

8.2. Ciclos definidos

Un ciclo definido es aquel en el que se conoce de antemano la cantidad de veces que se repetirá un bloque de instrucciones.

En C++, el ciclo más utilizado para este tipo de situaciones es:

- **for**

Se utiliza cuando el número de repeticiones es conocido antes de iniciar la ejecución del ciclo.

8.3. Ciclos indefinidos

Un ciclo indefinido es aquel en el que no se conoce previamente cuántas veces se repetirá un bloque de instrucciones.

La repetición depende del resultado de una condición lógica.

En C++, los ciclos utilizados para este tipo de situaciones son:

- **while**
- **do-while**

Estos ciclos continúan ejecutándose hasta que la condición deje de cumplirse.

9. Ejemplo

Escriba un algoritmo que imprima todos los números menores que un valor, iniciando como primer número en cero.

Ejemplo:

Si `valor = 13`, el algoritmo imprime:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

Realice una versión del algoritmo aplicando cada una de las estructuras iterativas anteriores.

10. Ejercicios

10.1. Ejercicio 1

Escriba un algoritmo que le solicite al usuario si desea continuar utilizando un programa. El algoritmo debe mostrar el siguiente mensaje:

`¿Desea continuar? S/N`

Si el usuario selecciona la letra **S** (en mayúscula), el programa continuará la ejecución y mostrará el conteo de las veces que el usuario ha digitado dicha letra.

Por otra parte, si el usuario selecciona la letra **N**, el algoritmo finaliza y muestra el siguiente mensaje:

`Hasta la próxima`

Utilice la estructura iterativa que desee.

10.2. Ejercicio 2

Escriba un algoritmo que le solicite al usuario si desea realizar las operaciones matemáticas de sumar, restar, multiplicar y dividir dos números. Ambos números deben ser solicitados al usuario; el usuario será quien decida cuál de las operaciones desea realizar.

El resultado se debe mostrar en pantalla con el siguiente formato:

`El resultado de la operación <operación>es <resultado>`

donde:

- `<operación>` puede ser: Suma, Resta, Multiplicación o División.
- `<resultado>` corresponde al valor obtenido al realizar la operación matemática seleccionada.

Agregue la opción de salir, realizada en el ejemplo anterior.

Recuerde utilizar las estructuras condicionales e iterativas vistas durante la tutoría, según corresponda.

10.3. Ejercicio 3

Antes de programar, es importante analizar el patrón que sigue una secuencia de números.

Determine cuál es el número que falta en cada una de las siguientes series. Una vez identificado el patrón, desarrolle un algoritmo que permita generar la serie correspondiente.

1. 2, 4, 6, 8, ____
2. 3, 6, 12, 24, 48, ____
3. 100, 50, 25, 12,5, ____
4. 2, 5, 10, 17, ____, 37
5. 1, 4, 9, 16, 25, ____
6. 1, 1, 2, 3, 5, 8, 13, ____

10.4. Ejercicio 4

Desarrolle un algoritmo en C++ que solicite al usuario ingresar un carácter.

Si el carácter corresponde a una vocal (a, e, i, o o u), el programa debe mostrar el siguiente mensaje:

El carácter es una vocal

De lo contrario, deberá mostrar:

El carácter no es una vocal

10.5. Ejercicio 5

Implemente un programa en C++ que determine si un estudiante aprobó o reprobó un examen, considerando que la nota mínima de aprobación es 70.

10.6. Ejercicio 6

Desarrolle un algoritmo en C++ que calcule el mínimo entre tres números ingresados por el usuario.

10.7. Ejercicio 7

Desarrolle un algoritmo en C++ que determine si un número es múltiplo de 5 y de 7 al mismo tiempo.

10.8. Ejercicio 8

Escriba un algoritmo en C++ que determine si una letra ingresada por el usuario es una vocal o una consonante.

10.9. Ejercicio 9

Escriba un algoritmo en C++ que solicite al usuario ingresar un número del 1 al 7, representando un día de la semana, y luego imprima el nombre del día correspondiente.

10.10. Ejercicio 10

Escriba un algoritmo en C++ que solicite al usuario ingresar un número del 1 al 12, representando un mes del año, y luego imprima el nombre del mes correspondiente.